

Revision Soutenance - Par Fonctionnalite

Ce document sert de plan de revision code pour la soutenance.

Objectif: comprendre rapidement comment les fichiers Frontend et Backend sont relies pour chaque fonctionnalite.

1. Vision globale de l'application

Stack technique

- Frontend: Angular standalone, HttpClient, Router, ng2-charts.
- Backend: FastAPI, SQLAlchemy, JWT, services metier.
- IA: RAG (Groq + Pinecone + embeddings HuggingFace), Vision pour panneaux.
- Base de donnees: MySQL.

Demarrage local (commandes)

```
# Backend
cd c:/securite-routiere-pfe/backend
c:/securite-routiere-pfe/.venv/Scripts/python.exe -m uvicorn app.main:app --reload

# Frontend
cd c:/securite-routiere-pfe/frontend/Plateforme-Securite-Routiere-master
npx ng serve

# Build verification
npx ng build --configuration development
```

2. Authentification (Register, Login, JWT, Roles)

But fonctionnel

- Creer un compte, se connecter, gerer session JWT, redirection selon role.

Fichiers Frontend

- src/app/auth/login/login.ts
- src/app/auth/register/register.ts
- src/app/core/services/auth.service.ts
- src/app/core/interceptors/auth.interceptor.ts

- src/app/core/guards/auth-guard.ts
- src/app/app.routes.ts

Fichiers Backend

- backend/app/api/auth.py
- backend/app/services/auth_service.py
- backend/app/models/user.py
- backend/app/schemas/auth.py

Liaison Front -> Back

- Login/Register appelle AuthService.
- AuthService appelle /api/auth/login ou /api/auth/register.
- Backend genere access token + refresh token.
- Front stocke tokens + user et l'interceptor ajoute Authorization Bearer.
- Les routes protegees utilisent authGuard cote front et get_current_user cote back.

Points a expliquer au jury

- Pourquoi access token court + refresh token long.
- Comment les roles admin/formateur/apprenant sont imposes.
- Que se passe-t-il si token expire.

3. Chatbot RAG (Texte)

But fonctionnel

- Repondre aux questions routieres FR/AR en s'appuyant sur des documents de reference.

Fichiers Frontend

- src/app/apprenant/chatbot/chatbot.ts
- src/app/core/services/chatbot.service.ts
- src/app/app.routes.ts (route apprenant/chatbot)

Fichiers Backend

- backend/app/api/chat.py
- backend/app/services/rag_service.py
- backend/app/services/language_detector.py
- backend/app/services/vector_store.py

- backend/app/services/document_loader.py
- backend/app/schemas/chat.py

Liaison Front -> Back

- Composant chatbot envoie la question via ChatbotService.askQuestion.
- Endpoint /api/chat/ask recoit la requete.
- rag_service:
 - detecte la langue,
 - cherche les chunks pertinents dans Pinecone,
 - construit le prompt,
 - appelle le modele Groq,
 - renvoie reponse + sources.
- Front affiche la reponse et les sources/panneaux detectes.

Points a expliquer au jury

- Difference LLM simple vs RAG.
- Pourquoi ingestion des documents est indispensable avant usage.
- Comportement en cas d'absence de contexte.

4. Detection panneau par image (Chatbot Vision)

But fonctionnel

- Envoyer une image et reconnaitre les panneaux.

Fichiers Frontend

- src/app/apprenant/chatbot/chatbot.ts (onImageSelected)
- src/app/core/services/chatbot.service.ts (detectSign)

Fichiers Backend

- backend/app/api/chat.py (endpoint /api/chat/detect-sign)
- backend/app/services/image_detector.py
- backend/app/services/sign_detector.py

Liaison Front -> Back

- Front envoie image multipart/form-data.
- Backend valide type/taille image.

- image_detector analyse via modele vision.
 - Matching avec la base de panneaux locale (sign_detector).
 - Retour: panneau, categorie, confiance, regles.
-

5. Documents + Ingestion (Admin)

But fonctionnel

- Uploader des documents puis les indexer dans la base vectorielle.

Fichiers Frontend

- src/app/admin/documents-management/documents-management.ts
- src/app/admin/chatbot-training/chatbot-training.ts
- src/app/admin/chatbot-training/chatbot-training.html

Fichiers Backend

- backend/app/api/documents.py
- backend/app/services/document_loader.py
- backend/app/services/vector_store.py

Processus exact

- Admin upload fichier dans /api/documents/upload.
- Fichier enregistré dans data/documents/fr ou data/documents/ar.
- Admin clique Lancer l'ingestion (/api/documents/ingest).
- Backend charge documents, split en chunks, calcule embeddings, push dans Pinecone.

Points a expliquer au jury

- Ordre obligatoire: Upload puis Ingestion.
 - Pourquoi clear/reindex peut etre necessaire.
 - Gestion des erreurs upload/timeout.
-

6. QCM (Creation, Liste, Passage, Resultats)

But fonctionnel

- Admin cree des QCM.
- Apprenant passe un QCM et obtient resultat.

Fichiers Frontend

- src/app/admin/qcm-create/qcm-create.ts
- src/app/admin/qcm-management/qcm-management.ts
- src/app/apprenant/qcm-list/qcm-list.ts
- src/app/apprenant/qcm-test/qcm-test.ts
- src/app/apprenant/qcm-resultat/qcm-resultat.ts
- src/app/core/services/qcm.ts

Fichiers Backend

- backend/app/api/qcm.py
- backend/app/models/qcm.py
- backend/app/schemas/qcm.py

Liaison Front -> Back

- Admin:
 - creer/modifier/supprimer/publication via endpoints admin qcm.
- Apprenant:
 - liste des QCM publies,
 - recuperation detail test,
 - soumission des reponses,
 - calcul score/passed,
 - historique perso.

Points a expliquer au jury

- Difference metier entre creation QCM et passage QCM.
- Comment est calcule le score.
- Comment fonctionne la publication/depublishation.

7. Categories QCM dynamiques

But fonctionnel

- Les categories ne sont plus hardcodees cote UI.

Fichiers Frontend

- src/app/core/services/qcm.ts
- src/app/admin/qcm-create/qcm-create.ts
- src/app/admin/qcm-management/qcm-management.ts
- src/app/apprenant/qcm-list/qcm-list.ts

Fichiers Backend

- backend/app/models/qcm.py (QCMCategory)
- backend/app/api/qcm.py (categories CRUD)
- backend/app/schemas/qcm.py

Points a expliquer au jury

- Avantage DB vs liste statique.
- Validation categorie active cote backend.

8. Dashboards dynamiques (Admin, Apprenant)

But fonctionnel

- Afficher indicateurs reels depuis base de donnees.

Fichiers Frontend

- src/app/admin/dashboard/dashboard.ts
- src/app/apprenant/dashboard/dashboard.ts
- src/app/core/services/statistics.ts

Fichiers Backend

- backend/app/api/statistics.py

Liaison Front -> Back

- Front appelle endpoints statistiques role-based.
- Backend calcule agregats SQL (users, cours, qcm, resultats).
- Front alimente cards + charts.

Points a expliquer au jury

- Pourquoi timeout/finalize ont ete ajoutes.
 - Comment vous avez resolu les blocages d'affichage.
-

9. Header dynamique et navigation par role

But fonctionnel

- Adapter les liens selon role connecte.

Fichiers Frontend

- src/app/shared/header/header.ts
- src/app/shared/header/header.html

Exemples

- Admin chatbot -> /admin/chatbot-training
 - Apprenant chatbot -> /apprenant/chatbot
 - Dashboard -> route selon role
-

10. Structure de revision (plan 3 jours)

Jour 1 - Backend

- Lire app/main.py puis api/*.py puis services/*.py.
- Verifier mentalement chaque flow de donnees.

Jour 2 - Frontend

- Routes -> composants -> services -> appels API.
- Refaire la navigation complete avec DevTools Network.

Jour 3 - Soutenance blanche

- Demarrer backend/frontend.
- Executer script demo complet:
 - 1) login admin
 - 2) upload document
 - 3) ingestion

- 4) test chatbot
 - 5) creation QCM
 - 6) passage QCM apprenant
 - 7) dashboards
-

11. Questions jury frequentes (et ce qu'il faut montrer)

- Comment garantisiez-vous la securite des routes ?
 - Montrer JWT + role checks.
 - Pourquoi votre chatbot est fiable ?
 - Montrer RAG, sources, ingestion maitrisee.
 - Comment reduire les erreurs d'UI ?
 - Montrer timeout/finalize et gestion loading/error.
 - Comment le systeme evolue ?
 - Montrer categories dynamiques, services modulaires, endpoints separables.
-

12. Check-list finale avant soutenance

- Backend lance sans erreur.
 - Frontend compile et sert.
 - Login admin et apprenant fonctionnent.
 - Upload + ingestion OK.
 - Chatbot texte + image OK.
 - QCM creation + publication + passage OK.
 - Dashboards affichent de vraies donnees.
 - Tu sais expliquer 1 flow complet en 90 secondes.
-

13. Fiche "Fichier par fichier" (explication detaillee)

13.1 Authentification

- src/app/auth/login/login.ts: gere l'ecran de connexion, recupere email/mot de passe et delegue au service

d'authentification.

- src/app/auth/register/register.ts: gere l'inscription, valide les champs cote interface et envoie les donnees au service.
- src/app/core/services/auth.service.ts: centralise toute la logique auth frontend (login/register/refresh/logout, stockage local, user courant).
- src/app/core/interceptors/auth.interceptor.ts: ajoute automatiquement le token Bearer aux requetes et tente un refresh en cas de 401.
- backend/app/api/auth.py: expose les endpoints auth officiels (/login, /register, /refresh, etc.).
- backend/app/services/auth_service.py: contient le coeur securite (hash bcrypt, creation/verification JWT, recuperation utilisateur courant).
- backend/app/models/user.py: definit la structure de l'utilisateur en base (role, statut actif, identifiants).

Points developpement importants (Auth)

- Toujours passer par AuthService cote front, jamais faire des appels HTTP auth disperses.
- En cas de bug d'accès, verifier dans l'ordre: token present -> interceptor -> role backend.
- Garder la logique de permission cote backend (le front ne suffit pas pour securiser).

13.2 Chatbot RAG texte

- src/app/apprenant/chatbot/chatbot.ts: gere l'interface conversation (envoi message, affichage reponse, feedback, historique, voix/image).
- src/app/core/services/chatbot.service.ts: sert de passerelle unique entre le frontend et les endpoints chatbot backend.
- backend/app/api/chat.py: recoit les requetes chat et appelle les services metier adequats (RAG, vision, feedback).
- backend/app/services/rag_service.py: orchestre le pipeline complet (langue -> retrieval -> prompt -> LLM -> reponse).
- backend/app/services/language_detector.py: detecte la langue et selectionne le prompt systeme adapte FR/AR.
- backend/app/services/vector_store.py: gere Pinecone et les embeddings, pour stocker et rechercher les chunks pertinents.
- backend/app/services/document_loader.py: charge les documents source et les decoupe en morceaux utilisables par le RAG.

Points developpement importants (Chatbot)

- En debug chatbot: verifier d'abord /api/chat/ask puis ingestion puis Pinecone.
- Si reponse vide/mauvaise: le probleme est souvent donnees source ou retrieval, pas UI.
- Expliquer au jury que RAG = controle des connaissances + traçabilité via sources.

13.3 Chatbot image (Vision)

- src/app/apprenant/chatbot/chatbot.ts: envoie l'image selectionnee et affiche le resultat d'analyse dans la conversation.

- backend/app/api/chat.py (route detect-sign): valide le fichier (format/taille) et lance le traitement vision.
- backend/app/services/image_detector.py: interroge le modele vision puis transforme l'analyse brute en sortie exploitable.
- backend/app/services/sign_detector.py: fait le mapping vers la base interne des panneaux (id, labels, categorie, image).

Points developpement importants (Vision)

- Toujours controler taille/type fichier en backend.
- Prevoir fallback propre quand aucun panneau n'est detecte.

13.4 Documents + Ingestion

- src/app/admin/documents-management/documents-management.ts: permet de lister, uploader et supprimer les documents d'entrainement.
- src/app/admin/chatbot-training/chatbot-training.ts: declenche l'ingestion et affiche les statistiques de l'index vectoriel.
- backend/app/api/documents.py: gere les endpoints admin documents (upload, delete, ingest, stats).
- backend/app/services/document_loader.py: extrait le texte depuis txt/pdf/docx puis prepare les chunks.
- backend/app/services/vector_store.py: indexe les chunks et sert la recherche semantique.

Points developpement importants (Ingestion)

- Ne pas oublier de relancer ingestion apres ajout/modif de document.
- Sur demo soutenance: montrer upload puis ingestion dans cet ordre.

13.5 QCM (creation -> passage -> resultat)

- src/app/admin/qcm-create/qcm-create.ts: gere la creation/edition d'un QCM complet (meta + questions + reponses).
- src/app/admin/qcm-management/qcm-management.ts: affiche la liste admin et les actions de publication/suppression.
- src/app/apprenant/qcm-list/qcm-list.ts: montre uniquement les QCM publies accessibles aux apprenants.
- src/app/apprenant/qcm-test/qcm-test.ts: gere le passage du test (selection reponses, timer, soumission).
- src/app/apprenant/qcm-resultat/qcm-resultat.ts: presente le score final et les details de correction.
- src/app/core/services/qcm.ts: centralise tous les appels API QCM et categories.
- backend/app/api/qcm.py: contient toute la logique API QCM (admin + apprenant + resultats).
- backend/app/models/qcm.py: definit les tables QCM, questions, reponses, resultats et categories.
- backend/app/schemas/qcm.py: valide et structure les payloads entres/sorties.

Points developpement importants (QCM)

- Bien separer flux admin (gestion) et flux apprenant (passage).

- La correction officielle se fait cote backend, jamais cote front uniquement.

13.6 Dashboards dynamiques

- `src/app/admin/dashboard/dashboard.ts`: charge les KPIs admin et alimente les graphes en temps reel.
- `src/app/apprenant/dashboard/dashboard.ts`: calcule les indicateurs apprenant a partir des stats, cours et resultats.
- `src/app/core/services/statistics.ts`: expose les appels statistiques par role.
- `backend/app/api/statistics.py`: effectue les agregations SQL et renvoie les indicateurs consolides.

Points developpement importants (Dashboards)

- Toujours prevoir timeout + finalize + fallback pour eviter ecran bloque.
- Expliquer la source de chaque KPI (requete SQL, formule, periode).

13.7 Header et navigation role-based

- `src/app/shared/header/header.ts`: calcule les routes dynamiques selon le role connecte (dashboard, chatbot, cours, qcm).
- `src/app/shared/header/header.html`: applique ces routes dans le menu desktop/mobile pour une navigation coherente.

Points developpement importants (Header)

- Mettre la logique de route dans `.ts` (fonctions), pas en dur dans HTML.
-

14. Bonnes pratiques developpement a citer en soutenance

14.1 Cote frontend

- Centraliser tous les appels API dans des services (auth, qcm, chatbot, statistics, cours).
- Utiliser timeout/finalize pour fiabiliser loading et eviter UI bloquee.
- Garder les composants concentres sur l'UI, pas sur la logique metier lourde.

14.2 Cote backend

- Mettre la logique metier dans services, endpoints minces dans `api/`.
- Valider toutes les entrees avec schemas Pydantic.
- Appliquer le controle role/permissions dans les endpoints proteges.

14.3 Front-Back integration

- Conserver des contrats stables (schemas) pour eviter les regressions.
- Tester chaque flux en vertical: UI -> service -> endpoint -> DB -> retour UI.

14.4 Commandes utiles pendant dev

```
# Front build rapide
cd c:/securite-routiere-pfe/frontend/Plateforme-Securite-Routiere-master
npm run build --configuration development

# Backend run
cd c:/securite-routiere-pfe/backend
c:/securite-routiere-pfe/.venv/Scripts/python.exe -m uvicorn app.main:app --reload

# Regenerer les PDF docs
cd c:/securite-routiere-pfe
c:/securite-routiere-pfe/.venv/Scripts/python.exe docs/generate_pdf.py
```